OpenMP — Lesson 2

Shared vs. private variables, and the data race

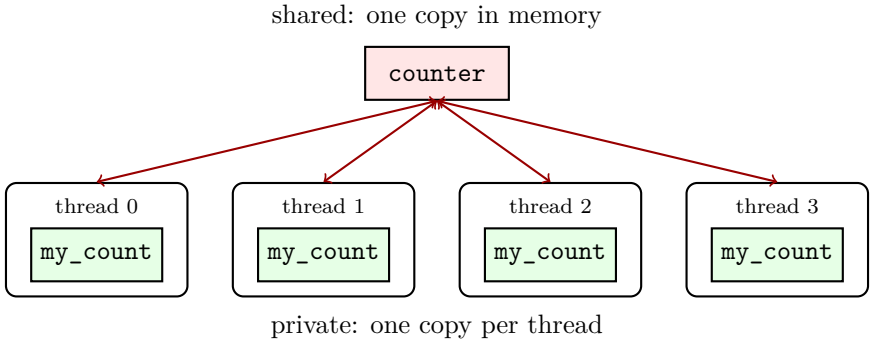
Study notes for Flor Vargas

November 2026

Sources: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the `shared` *and* `private` *clauses) and Ch. 7 "Troubleshooting" (data races).*

1 The idea

All threads of a team live in the **same address space**. That is what "shared-memory" programming means. So for every variable used inside a parallel region, we must know: is there *one* copy that all threads use (**shared**), or does each thread get *its own* copy (**private**)?



2 The rules

Variable	Default attribute
declared *before* the parallel region	**shared**
declared *inside* the region (incl. a `for` loop index declared there)	**private**
listed in a `private(...)` clause	**private**, new *uninitialised* copy
listed in a `shared(...)` clause	**shared** (makes the default explicit)

3 The data race

`counter++` looks like one operation but the CPU does three: *read* `counter` into a register, *add* 1, *write* it back. If two threads read the same old value before either writes, both write the same new value, and one increment is lost:

time	thread 0	thread 1
1	read `counter` → 5	
2		read `counter` → 5
3	write 6	
4		write 6 (should be 7)

This is a **data race**: two or more threads access the same memory location, at least one of them writes, and nothing orders the accesses. The result depends on timing.

4 The code

shared_private.c

```c
/*
 * shared_private.c -- OpenMP Lesson 2: shared vs private variables.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (shared and private clauses), Ch. 7 (data races).
 *
 * Compile (no -O flag, so the compiler does not hide the race):
 *     gcc -Wall -fopenmp -o shared_private shared_private.c
 * Run:
 *     OMP_NUM_THREADS=4 ./shared_private
 */

#include <stdio.h>
#include <omp.h>

#define N 1000000    /* increments done by EACH thread */

int main(void)
{
    int counter  = 0;   /* declared BEFORE the region -> SHARED by default:
                           all threads read and write the same memory cell. */
    int my_count = 0;   /* also declared before, but we will make it
                           PRIVATE with a clause below.                     */
    int nthreads = 1;

    #pragma omp parallel private(my_count)
    {
        /* private(my_count): each thread gets its OWN my_count.
           The private copy starts UNINITIALISED, so set it here.          */
        my_count = 0;

        for (int i = 0; i < N; i++) {
            counter++;      /* SHARED: read, add 1, write back.
                               Two threads can read the same old value
                               and one increment is lost -> DATA RACE.    */
            my_count++;     /* PRIVATE: no other thread touches it -> safe */
        }

        printf("  thread %d: my_count = %d\n", omp_get_thread_num(), my_count);

        if (omp_get_thread_num() == 0)
            nthreads = omp_get_num_threads();   /* only one thread writes it */
    }

    printf("expected counter = %d\n", nthreads * N);
    printf("actual   counter = %d\n", counter);
    return 0;
}
```

```
gcc -Wall -fopenmp -o shared_private shared_private.c
OMP_NUM_THREADS=4 ./shared_private
```

Typical output (your numbers will differ):

```
  thread 1: my_count = 1000000
  thread 3: my_count = 1000000
  thread 0: my_count = 1000000
```

```
  thread 2: my_count = 1000000
expected counter = 4000000
actual   counter = 2297568
```

Key ideas

- Every `my_count` is exactly 10^6: private data is always safe.
- `counter` is wrong and changes from run to run: a shared variable that several threads *write* is a race unless something coordinates them.
- Reading shared data is fine. The problem is concurrent *writes*.
- `private(x)` gives an uninitialised copy. Always assign it inside the region.
- `nthreads` is shared but written by only one thread, so there is no race.

5 The experiment

Experiment

Question: when does the race appear, and is it reproducible?

1. Run with 1, 2 and 4 threads:

```
for t in 1 2 4; do echo "== $t threads"; OMP_NUM_THREADS=$t ./shared_private; done
```

2. Run the 4-thread case five times and compare the last line:

```
for r in 1 2 3 4 5; do OMP_NUM_THREADS=4 ./shared_private | tail -1; done
```

3. Change `#define N 1000000` to `#define N 100`, recompile, and repeat step 2.

What to observe and answer.

1. Why is the result always correct with 1 thread?
2. Why does the wrong value change between runs?
3. With `N 100` the result is usually correct. Does that mean the program is correct?

Answers (read after trying). (1) With one thread there is nobody to race with: the read–add–write sequences never interleave. (2) How often two threads interleave their read and write depends on scheduling and timing, which change every run. This is why races are hard to debug. (3) No. With few iterations each thread finishes before the next one even starts, so the accesses rarely overlap. The race is still there and can appear at any time. A program that is "usually right" is wrong. Never trust a parallel result just because one test passed.

Next lesson. How to update a shared variable safely: `critical`, `atomic`, and the `reduction` clause, which is the right tool for this counter (Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.